

Documentação Técnica e de Desenvolvimento do Portal Saúde Fácil (TCC)

Este documento apresenta a fundamentação técnica, arquitetura de software, modelagem de dados e descrição dos módulos funcionais desenvolvidos para o **Portal Saúde Fácil**. Ele foi estruturado sob rigor acadêmico para servir de base direta para a seção de **Desenvolvimento / Engenharia do Sistema** do seu Trabalho de Conclusão de Curso (TCC).

1. Arquitetura de Tecnologia e Linguagens de Programação

O sistema adota uma arquitetura cliente-servidor descentralizada, focando em portabilidade, alta performance visual e facilidade de implantação local. A divisão de responsabilidades tecnológicas é descrita a seguir:

```
graph TD
    Client[Navegador Web / Frontend] -- HTTP Requests/JSON --> Server[Servidor Flask / Backend]
    Server -- SQL Queries --> DB[(SQLite Database)]
    Client -- Cache --> Storage[(sessionStorage / localStorage)]
```

1.1. Frontend (Interface do Usuário)

- **HTML5 (Estrutura)**: Utilizado para definir a estrutura semântica das páginas, garantindo acessibilidade e indexação (SEO). Todos os modais de autenticação, wizards de atendimento e grades informativas utilizam elementos semânticos estruturados.
- **CSS3 (Estilização e Animações)**: Responsável pela identidade visual premium. Aplica conceitos modernos de **Glassmorphism** (fundos translúcidos com `backdrop-filter: blur`, sombras complexas e bordas semitransparentes), layouts responsivos adaptados via CSS Grid e Flexbox, e micro-animações (como rotações tridimensionais no carrossel de notícias e elevações no hover dos cards).
- **JavaScript (ES6+ - Lógica de Cliente)**: Controla toda a reatividade do sistema de forma nativa (Vanilla JS), eliminando dependências pesadas. Gerencia requisições assíncronas (via `Fetch API`), manipulação dinâmica do DOM, controle de estado local (armazenamento temporário em `sessionStorage` e persistente em `localStorage`), e a renderização interativa dos carrosséis 3D e do mapa de doenças.

1.2. Backend (Servidor de Aplicação)

- **Python 3**: Linguagem escolhida para o backend devido à sua robustez, legibilidade e rica biblioteca padrão para desenvolvimento ágil.
- **Flask (Microframework)**: Utilizado para expor os endpoints da API RESTful e servir arquivos estáticos. O Flask gerencia o roteamento de páginas (como `/admin`, `/painel-medico`, etc.), sessões de usuário (`flask.session`) para segurança nas requisições, e middlewares para tratamento de CORS e uploads

de arquivos.

1.3. Banco de Dados

- **SQLite3:** Banco de dados relacional embutido de alto desempenho. Escolhido pela portabilidade extrema (toda a estrutura e registros são armazenados em um único arquivo físico `database.db`), dispensando a necessidade de configuração e manutenção de servidores de banco de dados externos durante a homologação do TCC.

2. Modelagem do Banco de Dados (SQLite)

O esquema relacional é estruturado para garantir a integridade dos dados e suportar o relacionamento entre múltiplos papéis (roles) de usuários. O banco inclui as seguintes tabelas principais:

2.1. Tabela `usuarios`

Armazena a identificação básica de todos os indivíduos do ecossistema (pacientes, médicos, enfermeiros, administradores e profissionais de TI).

- `id` (INTEGER, Chave Primária): Identificador único do usuário.
- `nome` (TEXT): Nome completo do usuário.
- `cpf` (TEXT, Único): Documento de CPF utilizado como chave única de login.
- `nascimento` (TEXT): Data de nascimento.
- `tipo` (TEXT): Define a role/papel do usuário (`paciente`, `medico`, `enfermeiro`, `admin`, `ti`).
- `senha` (TEXT): Hash/senha de autenticação do usuário.
- `sus` (TEXT): Número do Cartão Nacional de Saúde (apenas para pacientes).
- `imagem` (TEXT): Caminho da foto de perfil.

2.2. Tabela `medico\_info`

Entidade especializada que estende a tabela `usuarios` sob relacionamento de 1-para-1 para os profissionais médicos.

- `usuario_id` (INTEGER, Chave Estrangeira): Referência ao `id` na tabela `usuarios`.
- `crm` (TEXT): Registro profissional no Conselho Regional de Medicina.
- `especialidade` (TEXT): Especialidade médica do profissional.
- `atende_telemedicina` (INTEGER): Flag binária indicando aptidão para teleatendimento (0 ou 1).
- `tipo_atendimento` (TEXT): Modalidade (`presencial`, `telemedicina` ou `ambos`).
- `presencial_ativo` (INTEGER): Estado de plantão do médico na unidade física.

2.3. Tabela `consultas`

Entidade de relacionamento muitos-para-muitos entre pacientes e médicos, registrando os agendamentos.

- **id** (INTEGER, Chave Primária): Identificador do agendamento.
- **paciente_id** (INTEGER, Chave Estrangeira): Referência ao paciente.
- **medico_id** (INTEGER, Chave Estrangeira): Referência ao médico.
- **data e hora** (TEXT): Data e horário agendados.
- **especialidade** (TEXT): Especialidade da consulta.
- **tipo** (TEXT): Modalidade (presencial ou telemedicina).
- **status** (TEXT): Estado da consulta (pendente, concluída, cancelada).
- **queixa** (TEXT): Sintomas ou queixa principal informada pelo paciente.

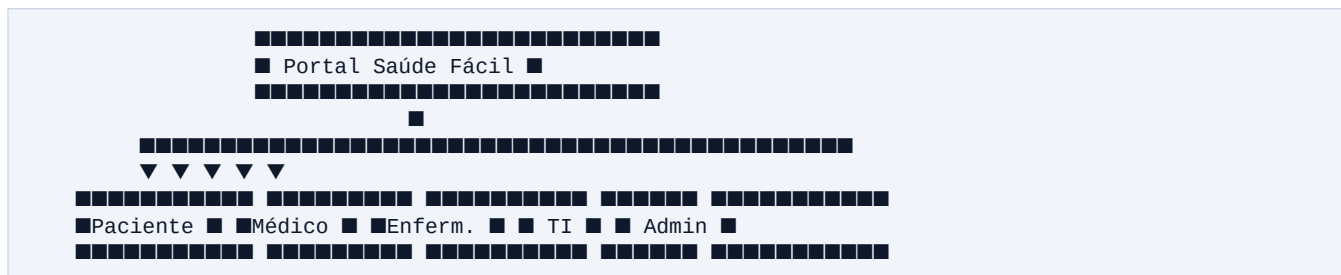
2.4. Tabela `settings`

Tabela chave-valor utilizada para armazenar parametrizações dinâmicas do sistema.

- **chave** (TEXT, Chave Primária): Nome do parâmetro (ex: `portal_titulo`, `portal_especialidades`).
- **valor** (TEXT): Conteúdo textual ou JSON estruturado do parâmetro.

3. Painéis do Sistema e Perfis de Acesso

O sistema é dividido em **5 painéis distintos**, garantindo que cada usuário tenha acesso restrito às funcionalidades compatíveis com seu cargo:



1. Painel do Paciente (Minha Área):

- Visualização de consultas futuras e histórico de atendimentos.
- Acesso rápido à sala de Telemedicina no momento agendado.
- Edição de dados de perfil (Nome, E-mail, Telefone, Cartão SUS e foto).

2. Painel do Médico:

- Visualização cronológica da agenda diária de consultas.
- Sistema de triagem e atendimento integrado de telemedicina.
- Acesso à ficha clínica e histórico médico do paciente.
- Emissão de receitas, atestados e formulários de encaminhamento para outras especialidades.

3. Painel de Enfermagem:

- Suporte à triagem física dos pacientes presenciais.
- Visualização de estatísticas de ocupação e atendimentos da unidade.

4. Painel do TI:

- Gerenciamento técnico e logs de depuração do sistema.
- Verificação do status de conexão dos serviços.

5. Painel Administrativo (Admin):

- Controle total de configurações do portal (títulos, banners e logomarcas).
- **Módulo de Especialidades:** CRUD completo para adicionar, editar ou remover especialidades médicas diretamente no banco de dados.
- Moderação de comentários públicos e envio de notificações globais.

4. Comunicação e Sincronização entre Painéis

Um dos requisitos de engenharia mais desafiadores do projeto é a **sincronização de dados em tempo real** sem acoplamento rígido. O fluxo de sincronização das especialidades médicas exemplifica este comportamento:

```
sequenceDiagram
    actor Admin
    participant DB as Banco SQLite
    participant API as Backend API
    participant Home as Home Page
    participant Agend as Tela Agendamento
    participant Tele as Tela Telemedicina

    Admin->>API: Adiciona nova especialidade (Urologia)
    API->>DB: Grava JSON em settings (portal_especialidades)

    Note over Home, Tele: Atualização imediata no carregamento
    Home->>API: settingsPublic()
    API-->>Home: Retorna lista de especialidades
    Home->>Home: Renderiza card de "Urologia" na Home

    Agend->>API: Carrega dropdown dinâmico
    Agend-->>Agend: Adiciona opção "Urologia"

    Tele->>API: Atualiza Passo 1 do Assistente
    Tele-->>Tele: Renderiza card "Urologia" no Wizard
```

- **Ponto Único de Verdade (Single Source of Truth):** A lista de especialidades médicas é cadastrada no painel Admin e guardada no banco (portal_especialidades).
- **Caching Inteligente de Sessão:** Para evitar requisições redundantes de rede que sobrecarreguem o servidor, as configurações públicas obtidas pela primeira vez são salvas no sessionStorage (cachedSettings).
- **Sincronização Horizontal:**
- A **Home Page** reconstrói sua grade de serviços no elemento .especialidades-grid.

- A **Tela de Agendamento** reconstrói o principal de marcação e o select secundário no formulário de cadastro de médicos (mapeando a seleção correta).
 - O **Wizard de Telemedicina** renderiza dinamicamente as opções do Passo 1 na classe `.grid-specialty`.
 - O **Painel do Médico** atualiza a caixa de encaminhamento de consultas (`#encaminh-especialidade`) automaticamente, permitindo encaminhar pacientes para as novas especialidades inseridas no sistema.
-

5. Fluxo de Autenticação (Login e Cadastro) e Segurança

- **Controle de Acesso Unificado (Gatekeeper):** O arquivo `home.js` e a rota backend `/api/login` realizam a identificação sob o protocolo de sessões HTTP seguras baseadas em cookie.
 - **Redirecionamento Inteligente (Login Lock):**
 - Caso um paciente tente interagir com um serviço que exija identificação (como clicar em um card de especialidade na Home), o sistema abre o modal de autenticação e salva temporariamente a especialidade desejada na sessão (`redirEspecialidade`).
 - Após o login ou cadastro bem-sucedido, o sistema lê esta variável de sessão e redireciona o usuário diretamente à página de agendamento com a respectiva especialidade pré-selecionada, evitando fricção e cliques desnecessários.
 - **Prevenção de Conflitos na Interface:** Tratamento inteligente via código para evitar conflitos de escopo de ID em elementos reutilizados em modais dinâmicos (como o dropdown `#especialidade` que aparece simultaneamente no agendamento e no formulário de novo cadastro de médico).
-

6. Módulos de Desenvolvimento

6.1. Módulo de Agendamento (``agendamento.html``)

Oferece um assistente para marcação de exames, vacinas ou consultas.

- **Filtros Dinâmicos de Médicos:** Ao selecionar uma especialidade e a modalidade (Presencial ou Telemedicina), o JavaScript dispara uma busca no endpoint `/api/medicos?especialidade=Nome`.
- **Resolução Genérica de Chaves:** O JavaScript lê as especialidades dinâmicas do cache e converte automaticamente a chave (ex: `clinico`) no nome textual reconhecido pelo banco (ex: `Clínica Geral`), eliminando estruturas de controle rígidas (`switch-case`).
- **Aviso no WhatsApp:** Gera um link formatado para que o paciente envie um lembrete automático da consulta via API de link direto do WhatsApp.

6.2. Módulo Mapa Dinâmico de Doenças

Focado na conscientização e educação em saúde pública.

- **Carrossel Vertical 3D:** Renderiza um controle rotativo vertical contendo as doenças cadastradas. Utiliza transformações de matriz 3D (`perspective(1000px) translate3d() rotateY() scale()`) para

dar sensação de profundidade aos cards de órgãos e vírus.

- **Design Premium e Fundo Transparente:** A seção possui um retângulo delimitador transparente (`border: 1px solid rgba(255, 255, 255, 0.25)`) que deixa transparecer o gradiente de cores da página principal de forma elegante.
- **Painel de Informações:** Ao girar o carrossel, o painel exhibe de forma reativa a gravidade da patologia, o especialista adequado a ser consultado e o encaminhamento inicial sugerido pelo protocolo do SUS.

6.3. Módulo de Campanhas de Saúde (`campanhas.html`)

Exibe banners informativos e cards sobre alertas epidemiológicos e programas de vacinação ativos.

- **Alinhamento Dinâmico:** As campanhas exibidas são gerenciadas de forma autônoma pelo painel do Administrador e atualizadas dinamicamente no frontend sem necessidade de alteração de código HTML.

6.4. Módulo de Dúvidas Frequentes (FAQ) (`duvidas.html`)

Seção de autoatendimento para pacientes.

- **Accordion Reativo:** Lista de perguntas e respostas colapsáveis estruturadas com animações de altura.
- **Busca em Tempo Real:** Uma barra de filtragem com detecção de digitação (`input event`) que oculta automaticamente os blocos de perguntas cujas palavras-chave não coincidam com a pesquisa do usuário.
- **Deep Linking:** Permite que o portal redirecione buscas feitas fora da página de FAQ (ex: da barra de pesquisa geral na Home) diretamente para a página de dúvidas, já filtrando o conteúdo com a palavra-chave informada nos parâmetros da URL.